

Модел

Model in ASP.NET Core MVC



Моделът създава едноименна таблица на БД със съответните свойства (properties) в нея.

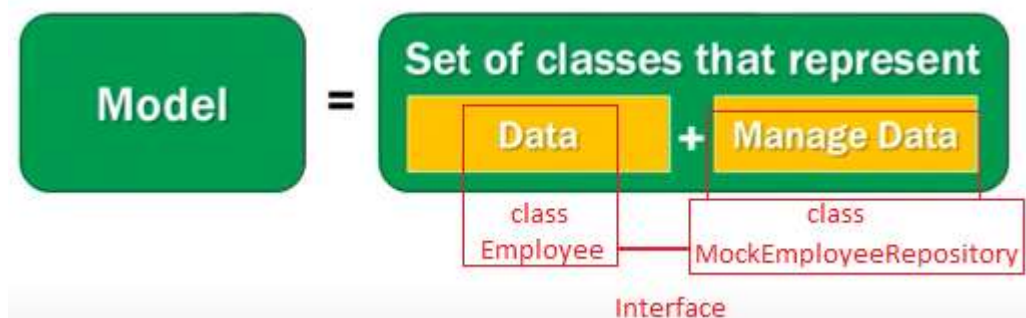
Искаме да извлечем данни от дадената БД-таблица и да ги изведем на екрана чрез уеб страница

Модел = данни + логика

=> представя + управлява

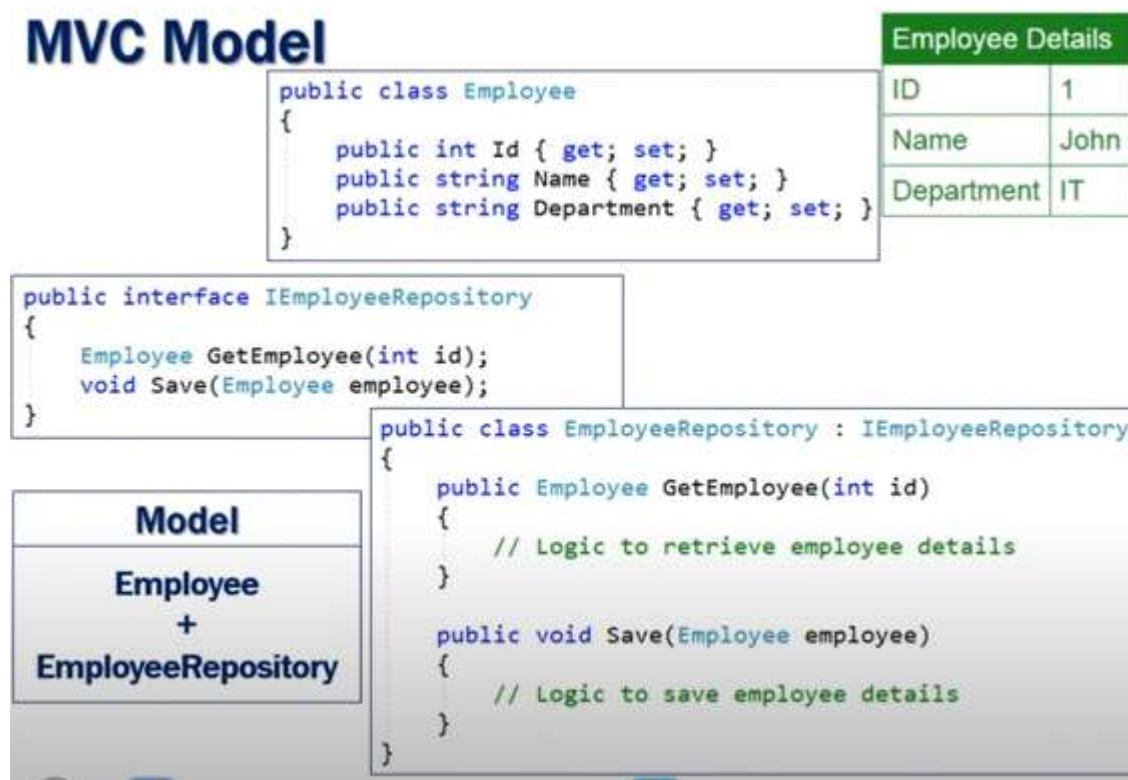
Модел = 1 клас за съхранение на данните (таблица БД) + 1 клас за логиката + 1 интерфейс на самата логика (абстракция) =>

модел = 2 класа + 1 интерфейс



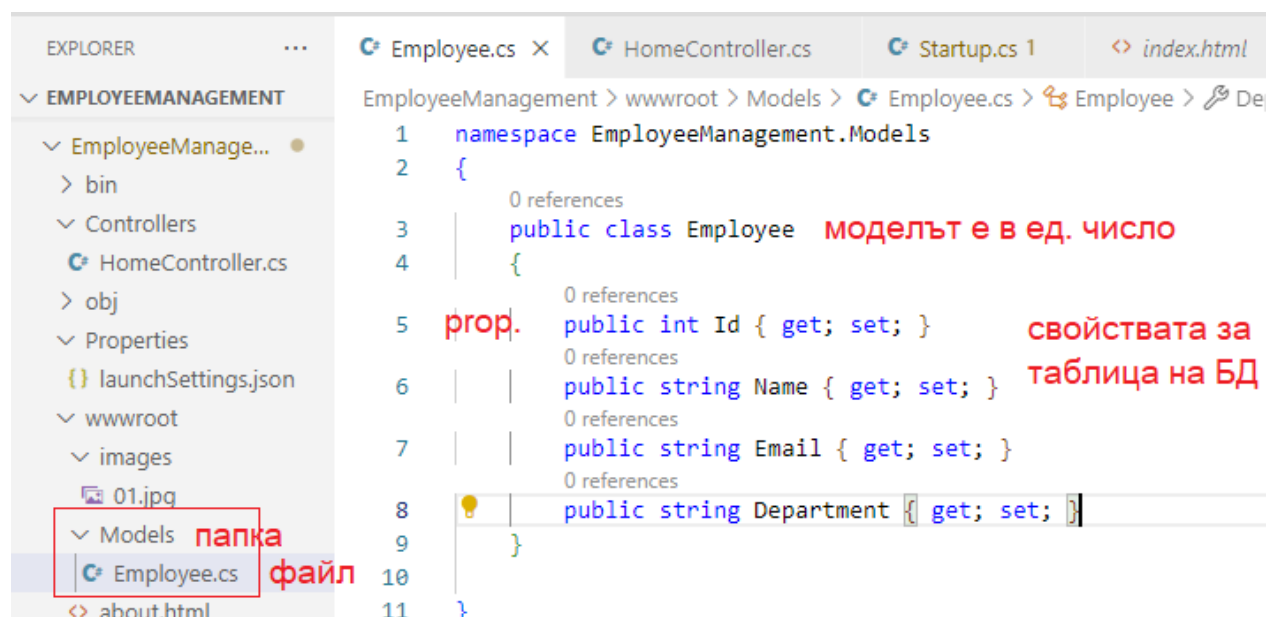
Извикваме интерфейса в HomeController

MVC Model



Създаваме си нова папка в wwwroot -> New Folder -> Models, а вътре в нея нов файл -> Employee.cs.

Не е задължително моделите и контролерите да са в едноименни папки, те могат да бъдат навсякъде в проекта, но е добра практика да се съхраняват по този начин за по-лесно управление.



Във Visual Studio автоматично написва скелет като

```
namespace EmployeeManagement.Models
```

```
{
    public class Employee
    {

    }
}
```

А ние само трябва да допишем това, което ни трябва вътре в него:

```
public int Id { get; set; }
public string Name { get; set; }
public string Email { get; set; }
public string Department { get; set; }
```

Във Visual Code, обаче, трябва всичко да си напишем сами.

ID, Name, Email, Department = property inside the class for DB-table.

Като последователност, ще разбием кода на 3 части: **БД – интерфейс – логика**, като съответно за целта използваме 3 модела:

Employee.cs – IEmployeeRepository.cs – MockEmployeeRepository.cs

Най-важен е MockEmployeeRepository, който ще извършва цялата работа и затова държи логиката. Бихме могли принципно да използваме само 1 модел (MockEmployeeRepository) без останалите и в него да напишем целия код без интерфейс, но тогава няма да може да използваме dependency injection и приложението ще бъде много трудно за промяна, управление и тестване. Затова, много по-добре е вместо на 1 място да се трупа много код, той да бъде разбит на няколко по-малки части, за да има по-добра видимост и управляемост.

Както вече казахме, за удобство, всички интерфейси започват с 'I' – IEmployeeRepository = InterfaceEmployeeRepository.

По същия начин в папка Models -> IEmployeeRepository.cs



```
namespace EmployeeManagement.Models
{
    public interface IEmployeeRepository
    {
        Employee GetEmployee (int Id);
        // declaration of method for get (create)
        // more methods: safe, update, delete
    }
}
```

Засега в интерфейса имаме само 1 метод за Get (който наподобява Create), но той ще бъде допълнен и с останалите CRUD (create, read, update, delete) операции като safe, update, delete.

В IEmployeeRepository няма почти никакъв код, но се използва междинно като интерфейс за по-голяма гъвкавост. Цялата работа ще се извършва от MockEmployeeRepository.cs.

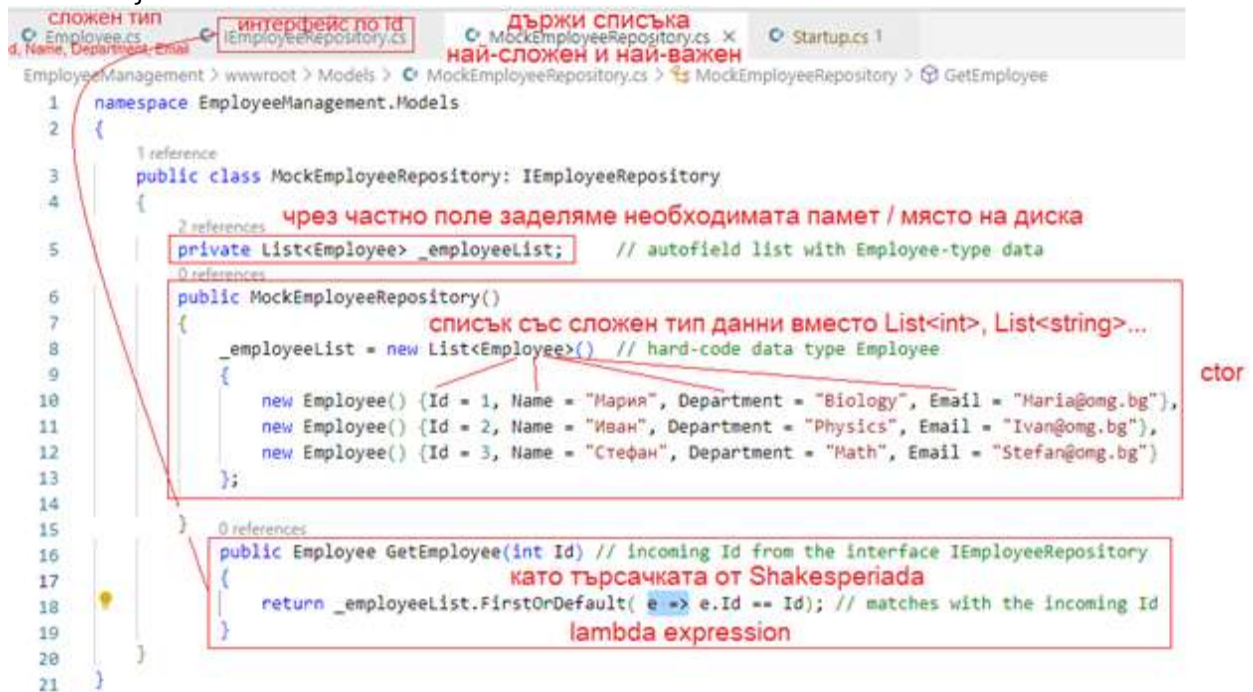
По същия начин създаване нашия 3-ти модел в папка Models -> MockEmployeeRepository.cs, който ще е най-важен и държи логиката. Той ще използва предишния модел с интерфейса, затова трябва да го извика с наследяване, т.е. с двуеточие. То означава, че ще вземе негови данни.

```
namespace EmployeeManagement.Models
{
    public class MockEmployeeRepository: IEmployeeRepository
    {
    }
}
```



Ще си създадем празен конструктор (той винаги взема автоматично същото име като модела MockEmployeeRepository). Клавишната команда е **ctor + Tab x 2**, за да го изпише вместо нас.

```
public MockEmployeeRepository()
{
}
```



Над конструктора заделяме частно поле за памет от мегабайти MB, т.е. казваме – задели необходимо дисково пространство, т.к. ще имам списък и той ще се използва.

```
private List<Employee> _employeeList; // autofield list
```

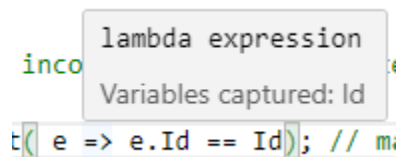
Списъка (List) по принцип се пълни с прост тип данни List<int>, List<string> и т.н., но ние сме ги комплектовали в сложен тип Employee, който държи няколко прости. По принцип първо се създава

новия лист, после автоматично с Ctrl + . се заделя необходимата памет, но логически в началото е по-добре да се обясни плавно.

Вътре в конструктора попълваме списъка с малко сурови данни – примерно, ако завършите ОМГ и работите в голяма научна компания с различни отдели в направления Биология, физика, математика:

```
_employeeList = new List<Employee>() // hard-code data type Employee
{
    new Employee() {Id = 1, Name = "Мария", Department = "Biology",
Email = "Maria@omg.bg"},
    new Employee() {Id = 2, Name = "Иван", Department = "Physics",
Email = "Ivan@omg.bg"},
    new Employee() {Id = 3, Name = "Стефан", Department = "Math",
Email = "Stefan@omg.bg"}
};
```

После извикваме интерфейса, защото искаме като подадем Id, интерфейсът да намери служителя и да ни върне цялата информация за него. Това става чрез търсачка, както правехме в Shakesperiada – чрез => което се нарича lambda expression за прихващане на Id.



```
public Employee GetEmployee(int Id) // incoming Id from the interface
{
    return _employeeList.FirstOrDefault( e => e.Id == Id); // matches
}
```

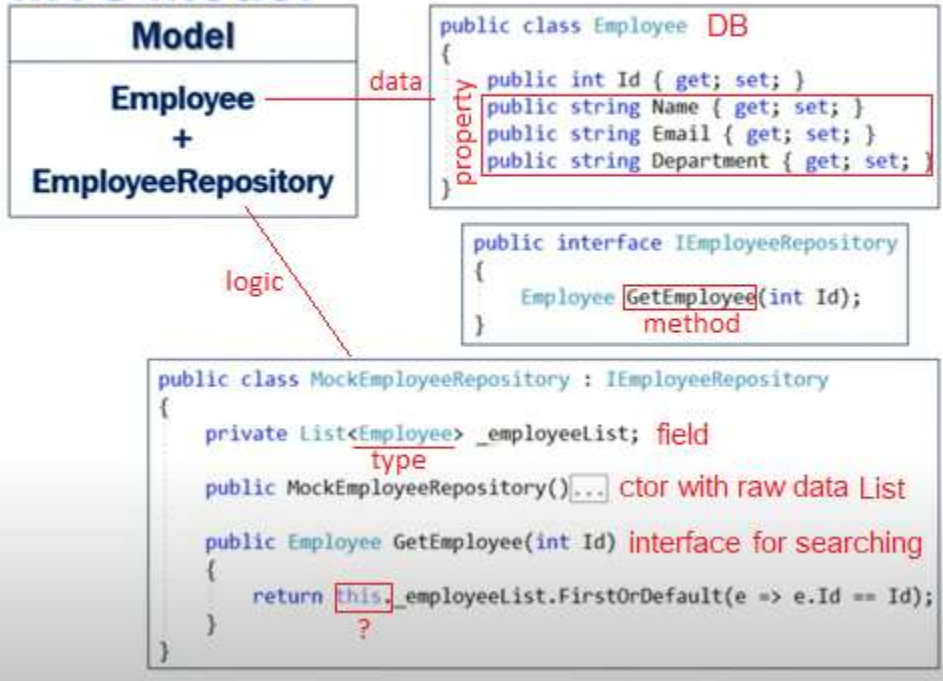
Връща резултат служителя от списъка, чието Id съвпада с подаденото Id.

Към момента, цялата работа се извършва от клас MockEmployeeRepository, който съхранява логиката.

Най-отгоре би трябвало да се добави автоматично библиотека
`using System.Collections.Generic;`

Имаме 1 модел, който държи свойствата (properties) за БД като сложен тип данни;
още 1 модел, който е интерфейс и само ни връща Id, който е за търсачка и CRUD операции;
последен модел, който използва предишните 2, за да създаде списък със сложен тип данни и при необходимост да върне резултат при поискване с Id – да върне всичко: Id, Name, Department, Email.

MVC Model



```
using System.Collections.Generic;
```

```
namespace EmployeeManagement.Models
```

```
{
    public class MockEmployeeRepository: IEmployeeRepository
    {
        private List<Employee> _employeeList; // autofield list
        public MockEmployeeRepository()
        {
            _employeeList = new List<Employee>() // hard-code data type Employee
            {
                new Employee() {Id = 1, Name = "Мария", Department = "Biology",
                Email = "Maria@omg.bg"},
                new Employee() {Id = 2, Name = "Иван", Department = "Physics",
                Email = "Ivan@omg.bg"},
                new Employee() {Id = 3, Name = "Стефан", Department = "Math",
                Email = "Stefan@omg.bg"}
            };
        }

        public Employee GetEmployee(int Id) // search with incoming Id
        {
            return _employeeList.FirstOrDefault( e => e.Id == Id);
        }
    }
}
```

} }